

Learning What We Never See

Expectation–Maximization at the Frontier

Lecture 5's **method**, expectation–maximization (EM), learns from data with a **hidden part**: it guesses the hidden part softly, then counts. It is behind clean training labels, chatbot search, image models, tokenizers and models that reason. Today you see it at the frontier, and try it yourself.

Yuwen Huang

Advanced Machine Learning and Deep Learning · HKUST (Guangzhou)

Builds on Lecture 5: Learning what we never observe

Outline

- ▶ **Learning what we never see (Lecture 5)**

Troubles 1–2: cleaning and searching data

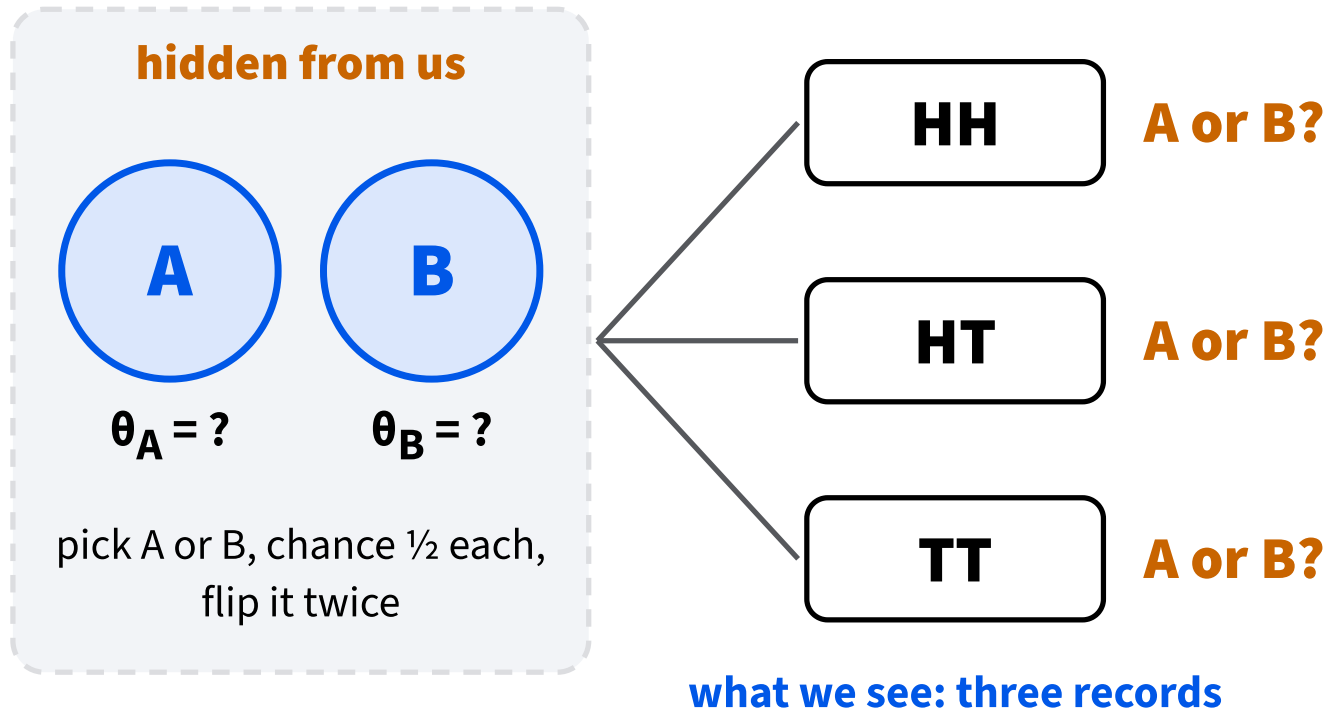
Troubles 3–5: inside image and language models

Conclusion and your turn

One story: the method of Lecture 5, expectation–maximization, meets five troubles at the frontier, and one small change fixes each. First: what does it do?

Learning what we never see

Two coins, one hidden: which coin made each record?



Look: each **record** is two flips of one coin. We see the flips but **never the coin**, and we do not know the heads chances θ_A, θ_B .

Q: Can we learn θ_A and θ_B anyway?

A: Yes. Give each record a chance for A and for B (a soft guess), count heads weighted by those chances, and repeat.

Learning what we never see: in three sentences

BACKGROUND

Data often hide one part of the story: which coin, which true label, which reasoning. We call it the **hidden variable**.

KEY IDEA

Guess the hidden part **softly** from the current model (the **E-step**), then **count** with those guesses (the **M-step**), and repeat.

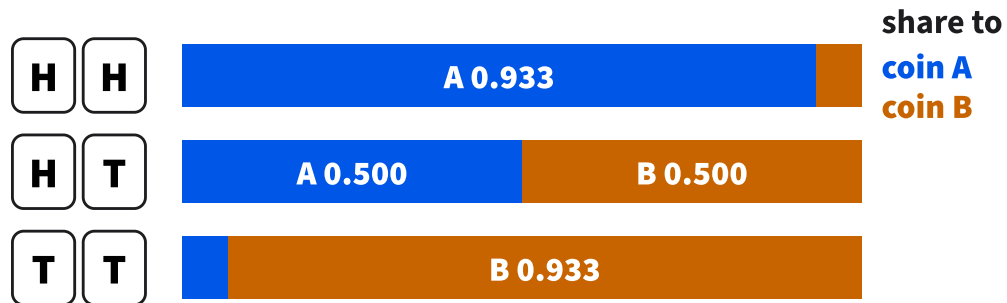
USED FOR

Every model with a hidden part, from groups of data points to the tokenizers of language models [Dempster, Laird and Rubin, 1977].

Learning what we never see

Main idea: guess each coin **softly**, **count** with the guesses, repeat.

round 15 • M-step: count



heads and flips, credited by the soft labels:

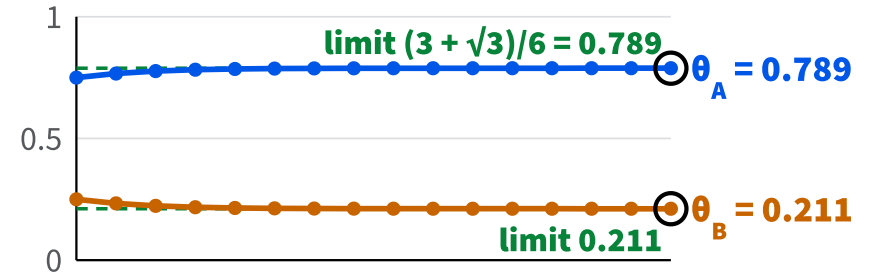
heads to A: $2 \cdot 0.933 + 1 \cdot 0.500 + 0 \cdot 0.067 = 2.366$

flips to A: $2 \cdot (0.933 + 0.500 + 0.067) = 3.000$

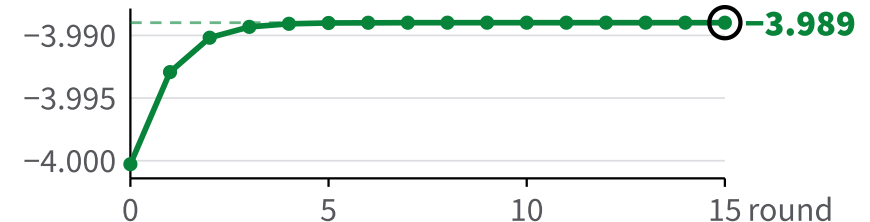
$\theta_A = 2.366 \div 3.000 = 0.789$

$\theta_B = 0.634 \div 3.000 = 0.211$

heads chances



log-likelihood $\ln p(\text{HH, HT, TT})$: never goes down



Watch: coin A makes HH 9 times likelier than B, so HH's **soft label** for A is 0.9; HT gets 0.5, TT 0.1 (**p. 30**). Counting with them, θ_A climbs $0.75 \rightarrow 0.767 \rightarrow \dots \rightarrow 0.789$, and the **likelihood**, the chance of all three records, **never goes down** (Lecture 5, Theorem 4).

Q: Why not give each record its most likely coin? A: HT is 50 : 50; a hard choice pushes one θ to 0 or 1, far too sure.

Try: another starting guess. Does EM always reach 0.789?

Main Question

**EM only needs soft guesses and counts.
Can it also decide whom to trust in a crowd, search a
million documents, learn a hidden description of each
image, cut words for language models, and teach a
model to reason?**

This is indeed the case, after a few small changes.

Keep this question in mind: each part changes the same method in one small way.

Learning what we never see

Six words for the rest of the lab.

Word	What it means today	Name in Lecture 5
record x	one thing we see: two flips, three people's labels, a word, a question with its answer	observed data x
hidden coin z	the part we never see: the coin, the true label, the cut of a word, the reasoning	hidden variable z
soft label	how much we believe each value of z given the record: its chance of making the record $\div$ the total chance (0.9 for A given HH)	posterior $p_{\theta}(z \mid x)$, the E-step
count	weighted heads $\div$ weighted flips (or right answers $\div$ answers), each record weighted by its soft label	M-step: expected counts $\div$ totals
round	one E-step, then one M-step	one EM iteration
likelihood	the chance of all records under the model; it never goes down from one round to the next	$p_{\theta}(x)$; Theorem 4

Read: every fix today keeps the two steps, **guess softly**, then **count**; only what is guessed or counted changes.

Learning what we never see

Five troubles at the frontier, five fixes.

	Trouble	Fix	Used for
1	whom to trust in a crowd	???	clean training labels
2	too much data	???	search for chatbots
3	the hidden part is many numbers	???	image models
4	nobody marks where to cut words	???	tokenizers
5	the reasoning is never written	???	models that reason

Read: troubles 1–2 appear when EM cleans and searches data, 3–5 inside image and language models. The fixes are revealed one by one.

Outline

Learning what we never see (Lecture 5)

- ▶ **Troubles 1–2: cleaning and searching data**

Troubles 3–5: inside image and language models

Conclusion and your turn

Next: two troubles that appear before any model is trained, when we clean the labels and search the data.

Trouble 1 in three sentences

- BACKGROUND** Training data are labelled by people, and people make mistakes; some make **more mistakes** than others.
- KEY IDEA** The true label is hidden, like the **coin**; each person's accuracy is unknown, like θ . EM finds **soft labels** for the photos and **whom to trust** together.
- USED FOR** Merging several people's noisy labels into one clean training set.

Trouble 1: whom to trust

Trouble: three people label photos (here 2 of 6), and they **disagree**.

	photo 1	photo 2
person 1	cat	dog
person 2	cat	dog
person 3	dog	dog
true label	?	?

Look: the **true label** of each photo is never given; it is the **hidden coin**. Each person's chance of being right, their **accuracy**, is unknown too.

Q: Why not just take the majority vote?

A: The vote trusts everyone equally. EM also learns who is careless, so a reliable minority can outweigh a careless majority.

Trouble 1: whom to trust

Fix: soft labels first; then each accuracy = expected right answers ÷ photos labelled.

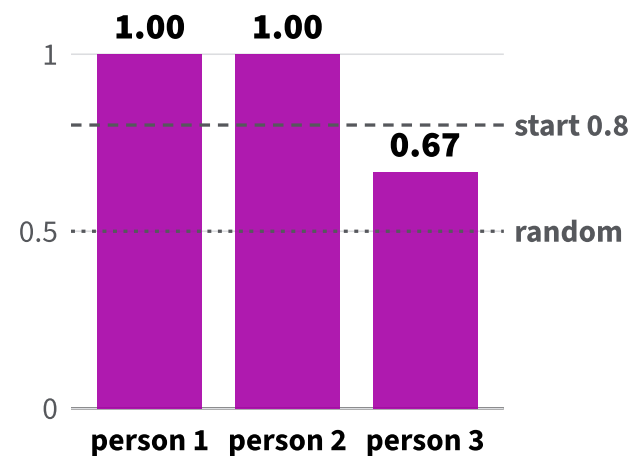
round 8 • M-step: accuracies

	person 1	person 2	person 3	soft truth
photo 1	cat	cat	dog	cat 1.000
photo 2	dog	dog	dog	dog 1.000
photo 3	cat	cat	cat	cat 1.000
photo 4	dog	dog	cat	dog 1.000
photo 5	cat	cat	cat	cat 1.000
photo 6	dog	dog	dog	dog 1.000

person 3: chance of being right on photos 1 ... 6, averaged:

$(0.000 + 1.000 + 1.000 + 0.000 + 1.000 + 1.000) \div 6 = 0.667$

accuracy: chance each person is right



persons 1 and 2 always agree:
EM ends up trusting them fully

Watch: all accuracies start at 0.8, so photo 1 gets the **soft label** 0.8 cat (p. 32). Scored against the soft labels, person 3, outvoted on photos 1 and 4, drops to **0.72**, then **0.67**; persons 1 and 2 end fully trusted.

Q: How can EM judge people without the truth? A: Those who agree with the others gain trust, and their votes count more.

Try: person 3 guesses at random: their accuracy falls to about 0.5. This is the one-coin version of **Dawid-Skene EM** [Dawid and Skene, 1979].

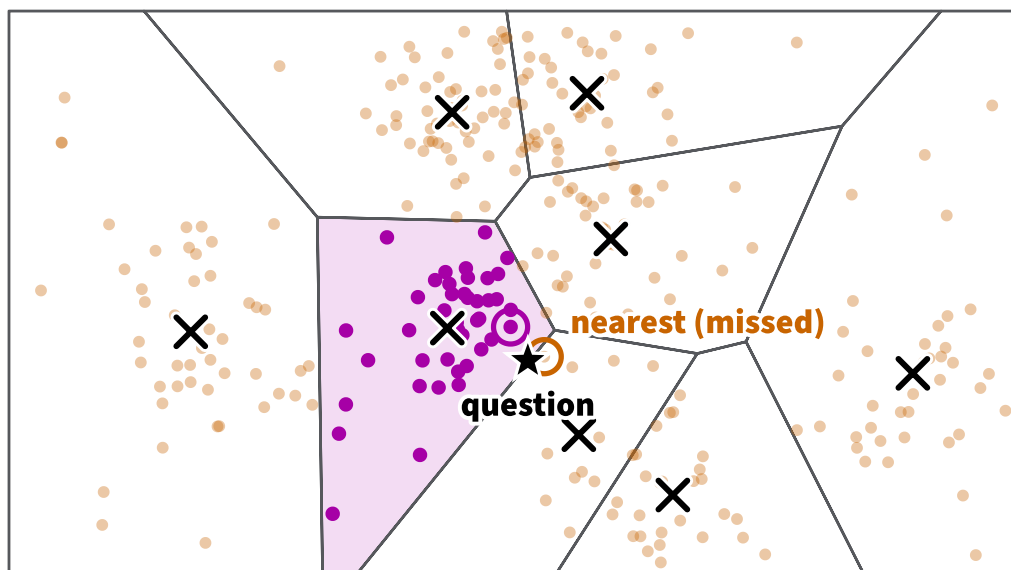
Trouble 2 in three sentences

- BACKGROUND** A chatbot stores each of millions of texts as a **point** (a list of numbers), with similar meanings close together. For each question it must find the **nearest** texts.
- KEY IDEA** **k-means** splits the points into groups (**cells**) around centres: it is EM with **hard labels**. A question then searches **one cell**, not the whole library.
- USED FOR** Vector-search indexes [Jégou, Douze and Schmid, 2011], such as those behind retrieval-augmented generation, where a chatbot answers from the texts it finds [Lewis et al., 2020].

Trouble 2: too much data

Trouble: millions of texts to compare. **Fix:** hard labels, search one cell.

texts as vectors (orange) · centres (black x)



search one cell (8 cells)

every text: 300

one cell: $8 + 41 = 49$

nearest text: missed x

magenta ring: best in the cell, not the nearest

over 50 questions: 4% of nearest texts missed,
about 50 distance computations each, not 300

Watch: model each group as a narrow bell-shaped cloud around a centre; then each point belongs fully to its nearest centre (Lecture 5, Theorem 5), and EM becomes **k-means**: file each point in its nearest cell, move each centre to the average.

Q: Why not compare the question with every text? A: A million distances; a thousand cells need about 2,000: 500 times fewer.

Try: more cells: fewer distances, more neighbours **missed**. This is **k-means vector search** [Lloyd, 1982; Jégou, Douze and Schmid, 2011].

Outline

Learning what we never see (Lecture 5)

Troubles 1–2: cleaning and searching data

► **Troubles 3–5: inside image and language models**

Conclusion and your turn

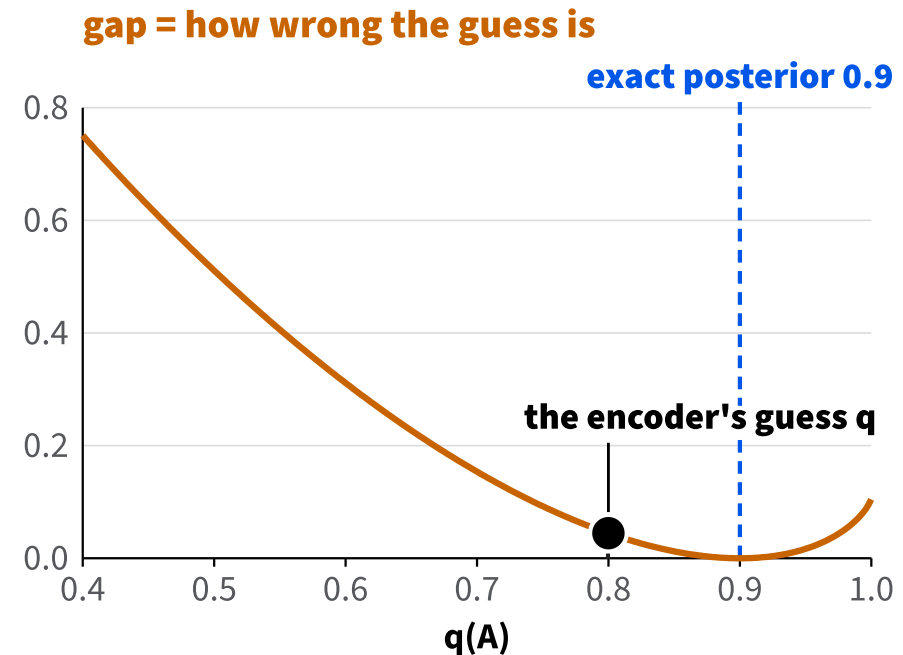
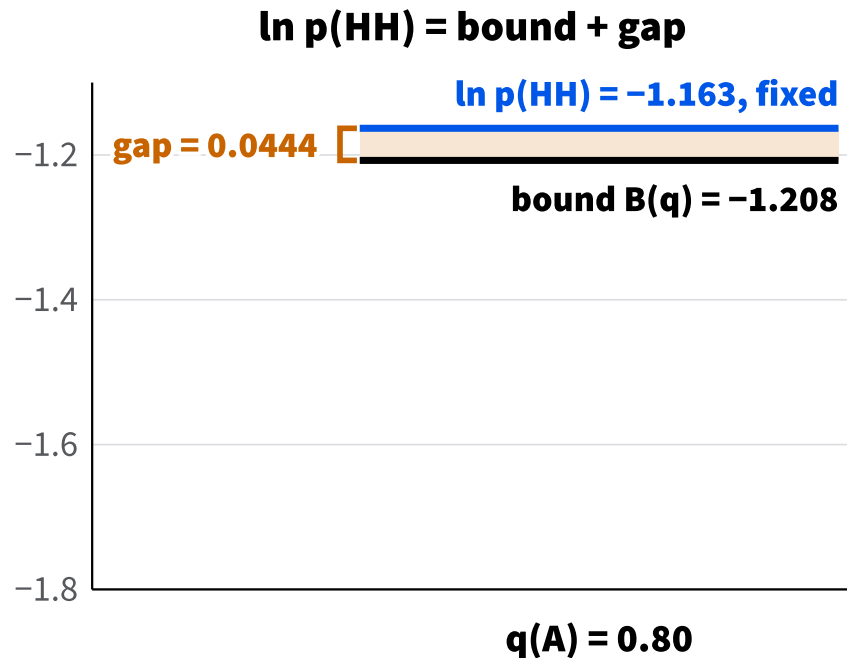
Next: the hidden coin becomes an image code, the cut of a word, and a whole line of reasoning.

Trouble 3 in three sentences

- BACKGROUND** An image model describes each image by a hidden **code** z , many real numbers such as pose and colour. The exact E-step is an integral over **all codes**, with no closed form.
- KEY IDEA** Train a network, the **encoder**, to output a guess of the soft label, and learn it together with the model.
- USED FOR** Image models that compress, generate and edit pictures.

Trouble 3: the code is many numbers

Trouble: infinitely many codes. **Fix:** an encoder learns the guess $q(z \mid x)$.



Watch: a coin stands in for an image. Any guess q gives a **bound**, a floor under $\ln p(x)$; the **gap** is zero only for the exact soft label (Lecture 5, Lemma 1). On HH, guessing 0.8 instead of 0.9 leaves a gap of **0.0444** (p. 33).

Q: Why not run the exact E-step? A: For an image it is an integral with no closed form; one pass of the encoder replaces it.

Try: move the guess to 0.9. This is the **variational autoencoder** [Kingma and Welling, 2014].

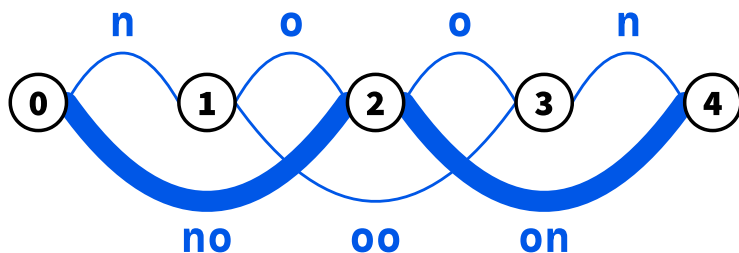
Trouble 4 in three sentences

- BACKGROUND** A language model reads text in pieces called **tokens**, such as un|happi|ness, but nobody marks where a word should be **cut**.
- KEY IDEA** Model a word as a row of independent pieces, each with its own chance. The cut is the **hidden coin**: EM gives each cut a soft label and counts **pieces** over **all cuts** at once.
- USED FOR** The tokenizers of language models such as T5 and ALBERT.

Trouble 4: where to cut words

Trouble: the **cut** is never given. **Fix:** soft labels for **all cuts**, then count pieces.

every path 0 → 4 is a cut of “noon”



thicker arc = likelier piece

**the cut no|on becomes
almost certain (0.999)**

E-step: share of each cut
= its chance ÷ chance of “noon”

n|o|o|n | 0.000

n|o|on | 0.001

n|oo|n | 0.000

no|o|n | 0.001

no|on | 0.999

M-step: piece probability
= expected count ÷ 2.00 pieces

n | 0.001

o | 0.001

no | 0.499

on | 0.499

oo | 0.000

Watch: with the pieces n, o, no, on, oo, “noon” has five cuts. Fewer pieces multiply fewer small chances, so no|on starts with **0.61** of the weight; counting raises **no** and **on**, and by round 4 no|on has 0.999.

Q: Why not list every cut? A: A 20-letter word has up to $2^{19} = 524,288$; forward-backward adds them letter by letter.

Try: favour “oo” at the start: EM sticks at n|oo|n. This is the **Unigram tokenizer** [Kudo, 2018], in the SentencePiece software.

Trouble 5 in three sentences

- BACKGROUND** Training data for reasoning hold **questions** and final **answers**; the reasoning between them, the **trace**, is **never written**.
- KEY IDEA** Let the model write **sample traces**, keep those that reach the right answer, and train on them; repeat.
- USED FOR** Teaching language models to reason step by step from answers alone.

Trouble 5: the hidden reasoning

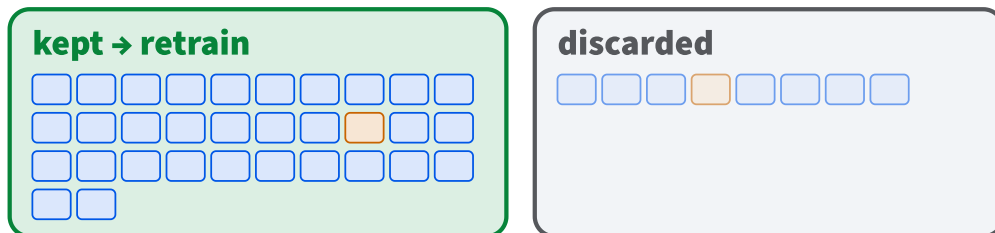
Trouble: the **trace** is never in the data. **Fix:** **sample** traces, keep the right ones, train.

the model answers 40 questions

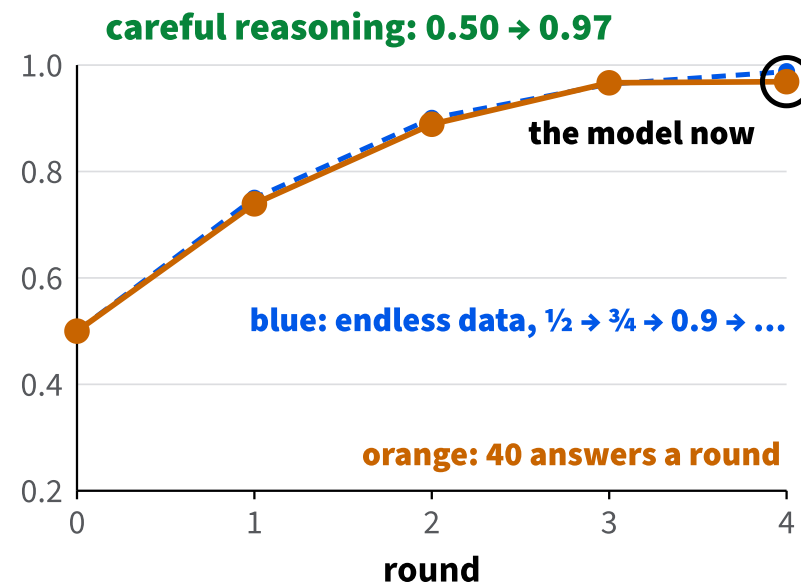
☐ careful: right $\frac{3}{4}$ ☐ hasty: right $\frac{1}{4}$

kept 32: 31 careful, 1 hasty

retrain: new $s = 31/32 = 0.969$



s = share of careful reasoning



Watch: a tiny model writes **careful** or **hasty** reasoning, half and half; careful is right more often. Keeping right answers keeps mostly careful traces: careful rises from $\frac{1}{2}$ to $\frac{3}{4}$ in one round (p. 34), about 0.97 by round 4.

Q: Why not keep every trace? A: Then careful stays at $\frac{1}{2}$: nothing is learned.

Try: keep all. This is STaR [Zelikman et al., 2022], recast as **Monte Carlo EM**, soft labels from samples [Phan et al., 2023].

Outline

Learning what we never see (Lecture 5)

Troubles 1–2: cleaning and searching data

Troubles 3–5: inside image and language models

► **Conclusion and your turn**

Finally: the five fixes side by side, what is still open, and what you can try yourself.

Conclusion

One **method**, five small changes, and the hidden labels are never observed.

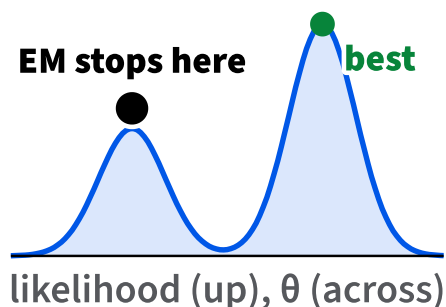
	Trouble	Fix	Used for
1	whom to trust in a crowd	soft labels, count right answers	clean training labels
2	too much data	hard labels: k-means cells	search for chatbots
3	the hidden part is many numbers	a learned guess: the encoder	image models
4	nobody marks where to cut words	add over all cuts	tokenizers
5	the reasoning is never written	sample traces, keep right ones	models that reason

Read: in every fix the two steps still alternate; the fix changes how the **E-step** guesses (hard, learned, over all cuts, sampled) or what the **M-step** counts (right answers).

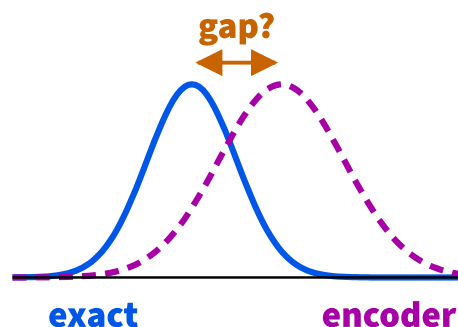
Conclusion

- ▶ It is possible to **learn what we never see**: guess the hidden part softly, then count (expectation–maximization).
- ▶ Each frontier changes how the **E-step** guesses, or what the **M-step** counts.
- ▶ We leave four **open problems**:

Stuck at a bad guess?



How wrong is the encoder?



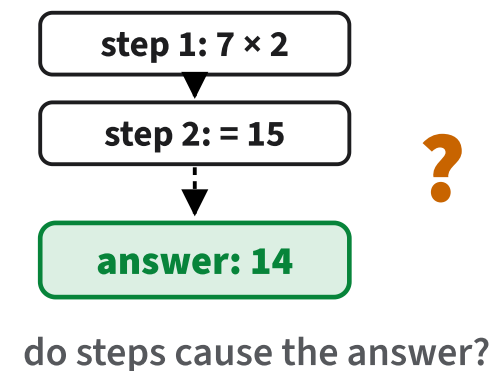
Better pieces than words?

unhappiness

un	happi	ness
unh	app	iness

which cut helps the model?

Is the reasoning honest?



Think: each would make a good course project; start from Jin et al. (2016) for 1, Cremer et al. (2018) for 2, Pagnoni et al. (2025) for 3 and Turpin et al. (2023) for 4.

Your turn

Predict first, then try: every animation in this lab has a knob.

In these slides

- ▶ the starting guess (p. 5): does EM always reach 0.789?
- ▶ a careless labeller (p. 12): how fast is trust lost?
- ▶ the number of cells (p. 14): speed or missed neighbours?
- ▶ the encoder's guess (p. 17): where is the **gap** zero?
- ▶ the starting pieces on “noon” (p. 19) and “keep all” (p. 21)

After class: `lab5_explore.py`

1. Two coins: start from $\theta = (0.6, 0.6)$. Where does EM go, and why?
2. A crowd of eight photos: what accuracy does EM give person 3?
3. k-means cells: how much faster is search, and what is missed?
4. A tiny reasoner: keep only right answers, or keep all?

Course site → Week 5 → Laboratory → Lab files. Each experiment needs one or two lines of Python.

Tip: write your prediction down before you click; the surprises are where the learning is.

Selected References I

Start here: Dempster, Laird and Rubin (1977) named the method and proved that the likelihood never goes down.



Cremer, C., Li, X., and Duvenaud, D. (2018).

Inference suboptimality in variational autoencoders.

In Proceedings of the 35th International Conference on Machine Learning, PMLR 80:1078–1086.



Dawid, A. P. and Skene, A. M. (1979).

Maximum likelihood estimation of observer error-rates using the EM algorithm.

Journal of the Royal Statistical Society, Series C (Applied Statistics), 28(1):20–28.



Dempster, A. P., Laird, N. M., and Rubin, D. B. (1977).

Maximum likelihood from incomplete data via the EM algorithm.

Journal of the Royal Statistical Society, Series B (Methodological), 39(1):1–38.



Jégou, H., Douze, M., and Schmid, C. (2011).

Product quantization for nearest neighbor search.

IEEE Transactions on Pattern Analysis and Machine Intelligence, 33(1):117–128.



Jin, C., Zhang, Y., Balakrishnan, S., Wainwright, M. J., and Jordan, M. I. (2016).

Local maxima in the likelihood of Gaussian mixture models: structural results and algorithmic consequences.

In Advances in Neural Information Processing Systems 29, pages 4116–4124.

Selected References II

Good entry points: Kingma and Welling (2014) for image models; Kudo (2018) for tokenizers.



Kingma, D. P. and Welling, M. (2014).

Auto-encoding variational Bayes.

In 2nd International Conference on Learning Representations; arXiv:1312.6114.



Kudo, T. (2018).

Subword regularization: improving neural network translation models with multiple subword candidates.

In Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics, pages 66–75.



Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., and Kiela, D. (2020).

Retrieval-augmented generation for knowledge-intensive natural language processing tasks.

In Advances in Neural Information Processing Systems 33, pages 9459–9474.



Lloyd, S. P. (1982).

Least squares quantization in pulse-code modulation.

IEEE Transactions on Information Theory, 28(2):129–137.

Selected References III

A short read: Zelikman et al. (2022), on how a model can teach itself to reason from answers alone.



Pagnoni, A., Pasunuru, R., Rodriguez, P., Nguyen, J., Muller, B., Li, M., Zhou, C., Yu, L., Weston, J., Zettlemoyer, L., Ghosh, G., Lewis, M., Holtzman, A., and Iyer, S. (2025).

Byte latent transformer: patches scale better than tokens.

In Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 9238–9258.



Phan, D., Hoffman, M. D., Dohan, D., Douglas, S., Le, T. A., Parisi, A., Sountsov, P., Sutton, C., Vikram, S., and Sauros, R. A. (2023).

Training chain-of-thought via latent-variable inference.

In Advances in Neural Information Processing Systems 36.



Turpin, M., Michael, J., Perez, E., and Bowman, S. R. (2023).

Language models don't always say what they think: unfaithful explanations in chain-of-thought prompting.

In Advances in Neural Information Processing Systems 36.



Zelikman, E., Wu, Y., Mu, J., and Goodman, N. D. (2022).

STaR: bootstrapping reasoning with reasoning.

In Advances in Neural Information Processing Systems 35, pages 15476–15488.

Thank you!

Next: Lecture 6, when even the E-step's guess must be approximated

Try EM yourself: the short Python version is on the backup slides that follow.

Backup: one round of Lecture 5

One round on HH, HT, TT from $\theta = (\frac{3}{4}, \frac{1}{4})$, each coin picked with chance $\frac{1}{2}$.

$$p(A | HH) = \frac{9/32}{9/32 + 1/32} = \frac{9}{9 + 1} = 0.9$$

$$p(A | HT) = \frac{3}{3 + 3} = 0.5, \quad p(A | TT) = \frac{1}{1 + 9} = 0.1$$

$$\text{heads for A} = 0.9 \cdot 2 + 0.5 \cdot 1 + 0.1 \cdot 0 = 2.3$$

$$\text{flips for A} = (0.9 + 0.5 + 0.1) \cdot 2 = 3$$

$$\theta_A = \frac{2.3}{3} = 0.767, \quad \theta_B = \frac{3 - 2.3}{6 - 3} = 0.233$$

E-step: $\frac{1}{2} \cdot \frac{3}{4} \cdot \frac{3}{4} = 9/32$ against $\frac{1}{2} \cdot \frac{1}{4} \cdot \frac{1}{4}$

the same rule

soft label $\times$ heads

soft label $\times$ flips

M-step: count $\div$ total

Notice: the M-step is the counting rule $\theta = \text{heads} \div \text{flips}$ of Lecture 5, with each record counted only as much as its soft label.

Backup: one round of Lecture 5

The whole method is a few lines of Python.

```
records = ["HH", "HT", "TT"]
a, b = 0.75, 0.25                                # starting guess: heads chances of coins A and B
for r in range(1, 31):
    heads = flips = 0.0
    for x in records:
        h, t = x.count("H"), x.count("T")
        pa, pb = a**h * (1-a)**t, b**h * (1-b)**t  # chance of the record from each coin
        s = pa / (pa + pb)                        # E-step: soft label for coin A
        heads += s * h; flips += s * (h + t)       # count with the soft label
    a, b = heads / flips, (3 - heads) / (6 - flips) # M-step: expected heads / expected flips
    if r in (1, 2, 30): print(r, round(a, 3), round(b, 3))
1 0.767 0.233
2 0.777 0.223
30 0.789 0.211
```

Try: start from $a = b = 0.5$ and explain what you see. More in `lab5_explore.py`.

Backup: Trouble 1

One round on the six photos: every accuracy starts at 0.8, cat and dog $\frac{1}{2}$ each.

$$p(\text{cat} \mid \text{cat}, \text{cat}, \text{dog}) = \frac{0.064}{0.064 + 0.016} = 0.8$$

| E-step, photos 1 and 4

$$p(\text{dog} \mid \text{dog}, \text{dog}, \text{dog}) = \frac{0.256}{0.256 + 0.004} = 0.9846$$

| E-step, photos 2, 3, 5, 6: all agree

$$a_1 = a_2 = \frac{0.8 + 0.8 + 4 \cdot 0.9846}{6} = 0.923$$

| expected right $\div$ photos

$$a_3 = \frac{0.2 + 0.2 + 4 \cdot 0.9846}{6} = 0.723$$

| person 3 is outvoted on photos 1 and 4

Notice: a_k is person k 's accuracy; it is counted exactly like $\theta_A = \text{heads} \div \text{flips}$, with “right answer” in place of “heads”.

Backup: Trouble 3

The gap of Lemma 1 for the guess $q(A) = 0.8$ on HH, where the exact soft label is 0.9.

Lecture 5, Lemma 1: $\ln p(x) = B(q, \theta) + \text{gap}$, and the gap is the Kullback–Leibler divergence from the guess q to the exact soft label:

$$\begin{aligned}\text{gap} &= 0.8 \cdot \ln \frac{0.8}{0.9} + 0.2 \cdot \ln \frac{0.2}{0.1} \\ &= 0.8 \cdot (-0.1178) + 0.2 \cdot 0.6931 \\ &= -0.0942 + 0.1386 = 0.0444\end{aligned}$$

guess against exact, for A and for B

the two logarithms

the bound is 0.0444 too low

Notice: with the exact guess $q(A) = 0.9$ both logarithms are $\ln 1 = 0$, so the gap is 0 and the bound touches $\ln p(x)$; that is why EM's E-step uses the exact soft label.

Backup: Trouble 5

The tiny reasoner: careful = coin A, hasty = coin B, “right” = heads.

It writes **careful** reasoning with chance $\pi = \frac{1}{2}$, right with chance $\frac{3}{4}$; else **hasty** reasoning, right with chance $\frac{1}{4}$.

$$p(\text{careful and right}) = \frac{1}{2} \cdot \frac{3}{4} = \frac{3}{8}, \quad p(\text{hasty and right}) = \frac{1}{2} \cdot \frac{1}{4} = \frac{1}{8} \quad \left| \text{sample, then check} \right.$$

$$\text{new } \pi = \frac{3/8}{3/8 + 1/8} = 0.75 \quad \left| \text{share of careful among kept} \right.$$

$$\text{keep all: } \pi = \frac{1/2}{1/2 + 1/2} = 0.5 \quad \left| \text{nothing is learned} \right.$$

Notice: the share of careful among the kept traces is the posterior of “careful” given “right”, the E-step of Lecture 5; training on them is the M-step.